

体系结构Lab6

PB22111665 胡揚嘉

实验目的

1. 理解 Attention 机制的原理，学会怎么使用普通办法实现 Attention。
2. 使用CUDA，实现分块实现 Attention 机制。
3. 理解 Attention 机制的并行化实现。

实验代码及分析

1. 实验代码

```
__global__ void Single_head_flash_atten(float *Q, float *K, float *V, float
*O, unsigned l, float softmax_scale){ //unsigned d,

    // 一个block处理结果O数组的AT行，总共有l/AT个block并行处理
    // 一个thread处理结果O数组的一行，对应自己Block的第i行
    // 每个thread重复矩阵计算l/AT次

    // 每个thread负责
    // 1. 将自己的Q那一行搬入share memory中
    // 2. 将本次循环中的自己的那一列K和那一列V搬入share memory中

    __shared__ float Qshare[AT][AD]; // 每个block的share memory中存储AT行Q
    __shared__ float Kshare[AD][AT]; // 每个block的share memory中存储AT列K
    __shared__ float Vshare[AT][AD]; // 每个block的share memory中存储AT行V

    // 中间结果寄存器
    float S[AT]; // 中间S矩阵的一行
    float Ssum = 0.0f; // S矩阵的一行的和
    float sum[AD]; // 结果O矩阵的一行

    unsigned i=blockIdx.x*blockDim.x+threadIdx.x; //当前线程处理的O数组的第i行
    unsigned index = threadIdx.x; // 当前线程在block中的位置

    // 1. 将自己的Q那一行搬入share memory中
    for(unsigned j = 0; j < AD; j++){
        Qshare[index][j] = Q[i*AD + j];
        sum[j] = 0.0f; // 初始化结果O矩阵的一行
    }

    for(unsigned loop = 0; loop < l/AT; loop++){ // 每个线程循环l/AT次
        // Ssum = 0.0f; // 每次循环前清空Ssum FIXME: 这里不需要每次循环都清空Ssum, 因为
        它是累加的
        // 2. 将本次循环中的自己的那一列K和那一列V搬入share memory中
        for(unsigned j = 0; j < AD; j++){
```

```

        kshare[j][index] = K[(index + loop*AT)*AD + j]; // K的第index+loop*AT
列
        vshare[index][j] = V[(index + loop*AT)*AD + j]; // V的第index+loop*AT
行
    }
    __syncthreads(); // 确保所有线程都完成了share memory的搬运
    // 3. 计算S矩阵
    for(unsigned j = 0; j < AT; j++){
        s[j] = 0.0f; // 初始化S矩阵的一行
        for(unsigned k = 0; k < AD; k++){
            s[j] += qshare[index][k] * kshare[k][j]; // Q*KT
        }
        s[j] = exp(s[j] * softmax_scale); // softmax
        ssum += s[j]; // 累加S矩阵的一行的和
    }
    // 4. 计算O矩阵的一行
    for(unsigned j = 0; j < AD; j++){
        for(unsigned k = 0; k < AT; k++){
            sum[j] += s[k] * vshare[k][j] ; // o[i][j] = s[i][k] * v[k][j] /
Ssum
        }
    }
    __syncthreads(); // 确保所有线程都完成了S和O的计算
}
// 5. 将结果写回O矩阵
for(unsigned j = 0; j < AD; j++){
    o[i*AD + j] = sum[j] / ssum; // 除以Ssum
}

return ;
}

```

分析：

1. 每个block处理结果O数组的AT行，总共有I/AT个block并行处理。
2. 每个thread处理结果O数组的一行，对应自己Block的第i行。并且每个thread计算本行的AD个位置。每个位置需要重复矩阵计算I/AT次，然后累加得到。
3. 每个thread负责：
 - 将自己的Q那一行搬入share memory中。
 - 将本次循环中的自己的那一列K和那一列V搬入share memory中。
 - 计算S矩阵的一行。然后与V矩阵计算得到O矩阵的一行的片段。
 - 循环I/AT次，完成O矩阵的一行的计算。
4. 使用 `__syncthreads()` 确保所有线程都完成了share memory的搬运和计算。
5. 注意softmax的计算需要除以Ssum，确保结果的正确性。
 1. 在代码中，softmax的计算是跨越不同的循环的。

实验结果和分析

```

(base) huyangjia@huyangjia:~/Coding/CS_Arch/comparch25spring-gem5/LAB6$ ./a.out
equal!
test pass!
l: 524288   time: 16729.421875000

```

首先是验证代码逻辑正确，结果显示正确

对于矩阵规模为 524288 的矩阵，消耗时间为：16.73s

实验报告问题

1. 请问原始flash attention 算法中能否将内层循环改为对K矩阵操作？请说明你变动后算法Q、K、V、O矩阵的搬运过程及这样做的好处与坏处（提示：考虑softmax）

1. 原始算法：

1. 搬运Q矩阵 $(N/T)^2$
2. 搬运K, V矩阵一共 $2 \cdot N/T$ 次
3. 搬运O矩阵一共 N/T 次。——每次结果不缓存，直接放入O矩阵中，放置缓存占用过多
4. 需要存储每行的softmax值从头到尾，显存占用大

2. 内循环改为对K矩阵操作：

1. 搬运Q矩阵 N/T 次
2. 搬运K, V矩阵一共 $2 \cdot (N/T)^2$ 次
3. 搬运O矩阵1次。——但是分 N/T 次逐个块的放入O矩阵中，每次放入的就是最后的正确值。
4. 每次对Q矩阵的外循环，可以清空之前行的softmax值。

3. 好处：

- 简化了 softmax 计算的复杂性，节省了中间结果记录的缓存代价。对整个注意力分数进行归一化，如果内层循环改为对K矩阵进行操作，会导致softmax计算变得更快

4. 坏处：

- 由上面的数据分析，当 $N \gg T$ 时（即当矩阵规模特别大），对矩阵的调度次数会更多而有上次实验可知，在GPU中，瓶颈在显存的带宽上，而这样的方式正好会导致加大带宽需要的作用。

所以由于带宽不足的可能性，会加大实际的 代码运行时间。

2. 原始flash attention算法中Q、K、V、O小块大小能如(Br,d/2)这样吗？请分析可能遇到的问题。

1. 逻辑上可以实现，相比目前的方法，遇到的问题如下：

- **归一化范围**：原始的 softmax 块需要考虑数据的范围就在一个线程中，在数次循环后，就可以得到需要的 s_{sum} ，用于计算中间矩阵S或者结果矩阵O

但是当一行的计算被切分成2份，然后使用不同的 thread 去负责，那么需要一个缓存通讯交换方式，使得不同的 thread 得知自己行的 softmax 需要的值，然后交给另外一个线程去将其处理

这样会碰见一下问题：

1. 数据共享问题：由于需要负责2半的2个 thread 都计算完毕后才能够进行后面的计算，所以需要一个机制去阻塞等待，这里会降低GPU的利用率。
2. 缓存问题：用于要阻塞等待，数据需要存储在缓存或者显存中，这会加大缓存需求，或者加大对带宽的压力

3. Single head Flash attention启动N个线程相比原始flash attention 算法对矩阵分块的复用有什么影响？请详细分析。

1. 对于 Single head Flash attention

存在重复搬运的问题：不同的 block 实际上都会搬运相同的K, V矩阵到 share mem 中，但是由于跨 block 了，只能每个块自行搬运一次。这加大了缓存存储代价和显存的带宽压力

同样的，每个 `Block` 需要搬运相同的KV矩阵，然后才能够开始计算，这会带来资源竞争问题，实际上会导致一些 `Block` 的计算速度延缓。

由上述分析可知，相比较原始计算机制：

- KV矩阵利用率：更低
- 全局的显存占用：更多
- 显存带宽和资源竞争：更大
- 计算并行性：更大

4. 你实现的Single head Flash attention运行速度比利用传统矩阵乘法实现的Single head Flash attention慢。请分析原因并给出改进方案及方案面临的困难。

1. 每一个不同的 `Block` 都需要搬运相同的KV矩阵块到自己的 `share mem`，这样会加大资源竞争，导致冲突和阻塞

解决办法：流水线化：

由划分：每个 `Block` 需要访问 N/T 次不同的KV矩阵块，总共有 N/T 个 `block`

那么恰好可以利用流水线：

- 第 i 次循环 ($i = [0, N/T-1]$),
 `Block` j ($j = [0, N/T-1]$) 去计算第 $(i+j) \% N/T$ 个KV矩阵块的值。

这样不同的线程块之间的访问的冲突 **极大的减小**

不足的是：由于没有冲突，显存带宽压力会增加，需要进行 `tradeoff`